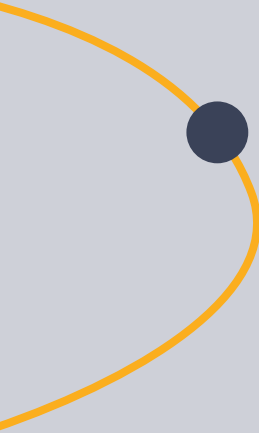


Paradigma de Desenvolvimento de Software Orientado a Objetos (OO)



Profa. Dra. Lina Garcés

Paradigma

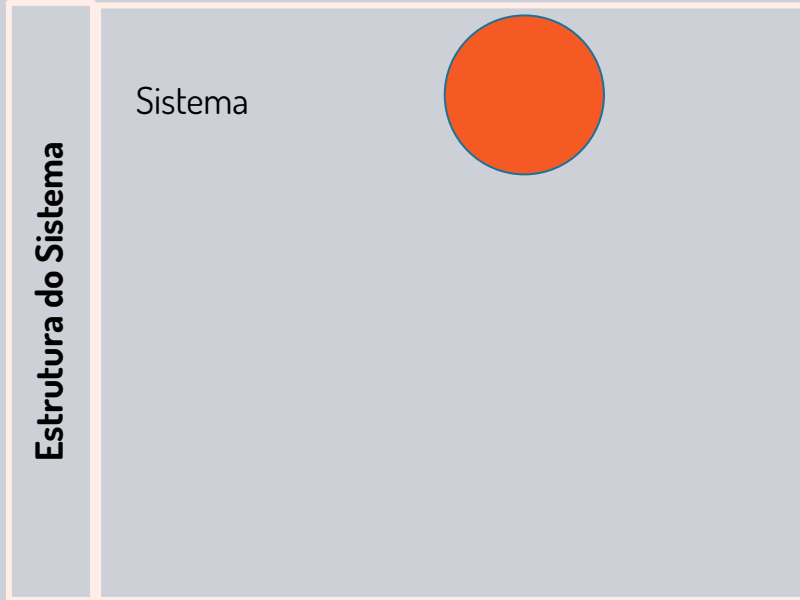
Conjunto de diretrizes que **orientam a criação de soluções** para problemas em uma área de conhecimento

Paradigma de OO

- Iniciou nos anos 60s
- Primeira linguagem OO proposta por Ole-Johan Dahl e Kristen Nygaard chamada Simula67
- C++ (evolução do C) popularizou POO nos anos 90s

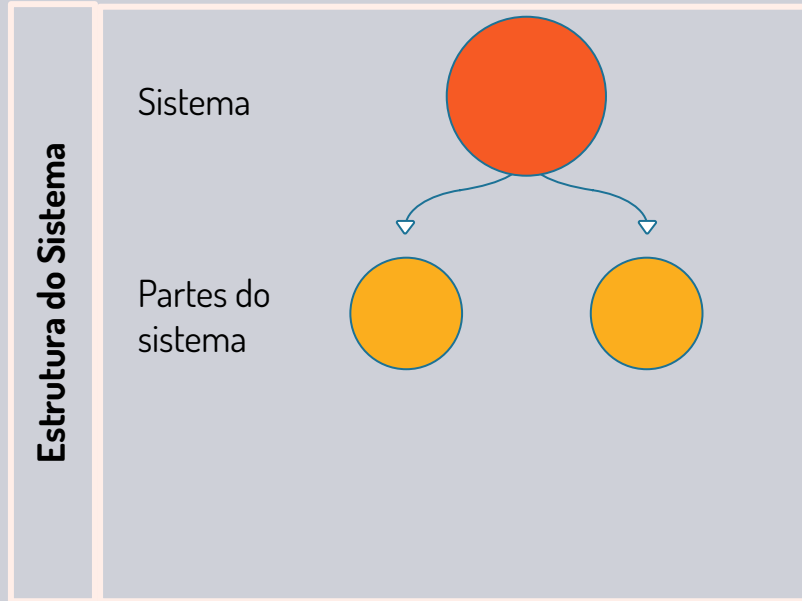
Paradigma de OO

Decomposição



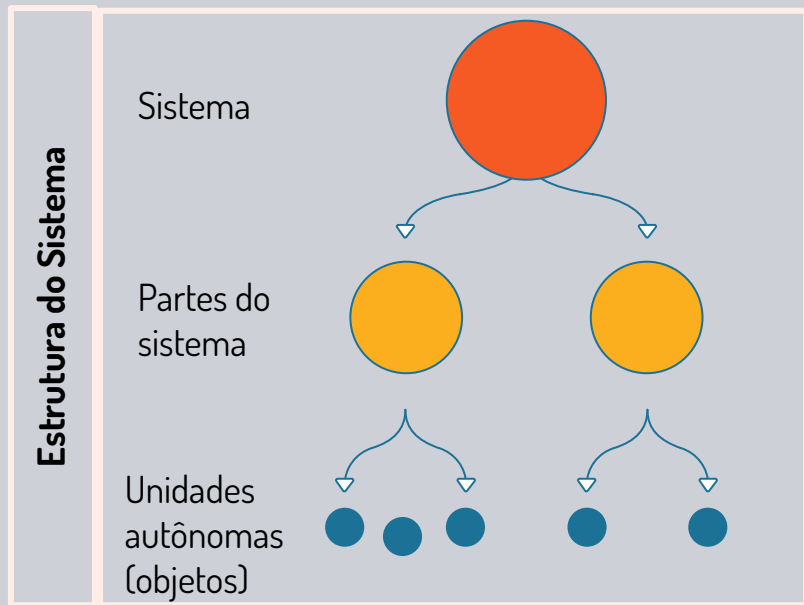
Paradigma de OO

Decomposição



Paradigma de OO

Decomposição



Paradigma de OO

Decomposição

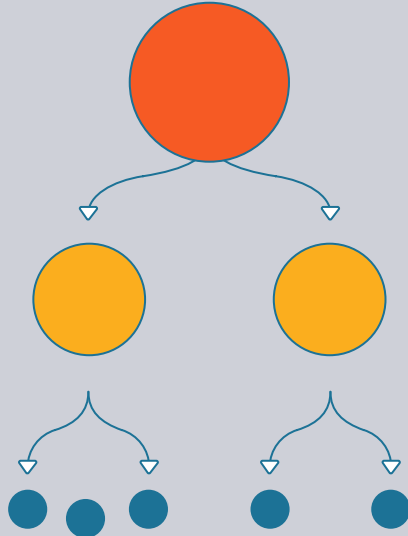
Composição

Estrutura do Sistema

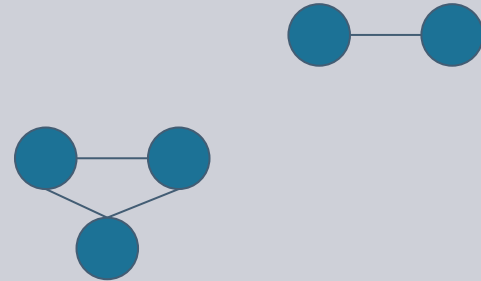
Sistema

Partes do sistema

Unidades autônomas (objetos)



Comportamento do Sistema



Paradigma de OO

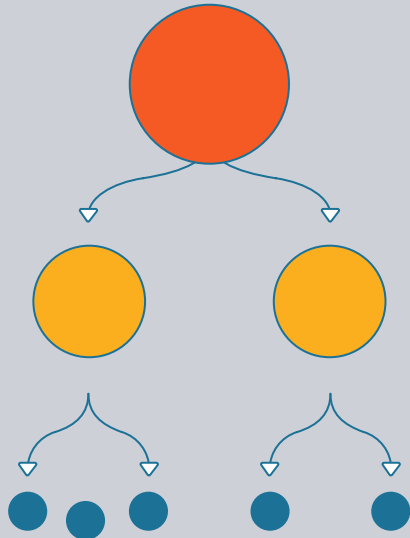
- Iniciou nos anos 70s
- Primeira linguagem OO proposta por Alan Kay (Matemático e Biólogo)

Estrutura do Sistema

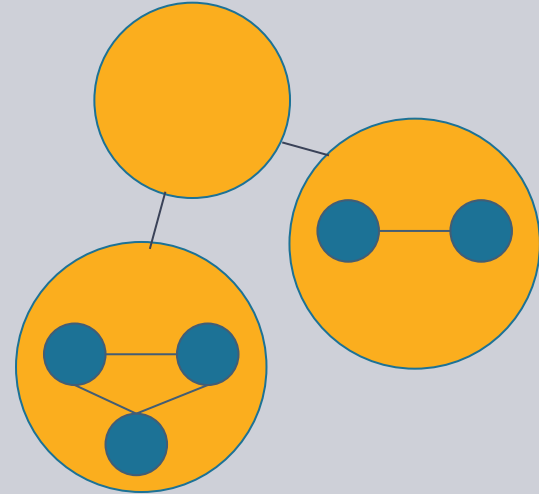
Sistema

Partes do sistema

Unidades autônomas (objetos)



Comportamento do Sistema



Paradigma de OO

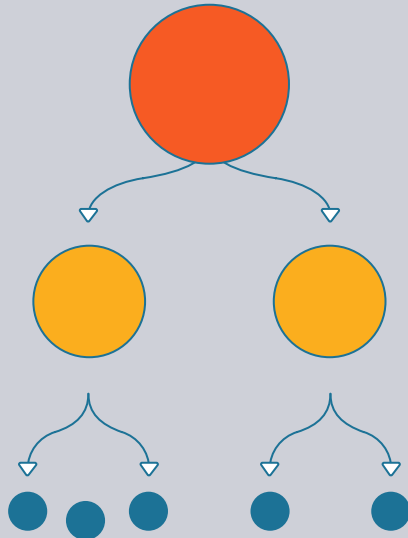
- Iniciou nos anos 70s
- Primeira linguagem OO proposta por Alan Kay (Matemático e Biólogo)

Estrutura do Sistema

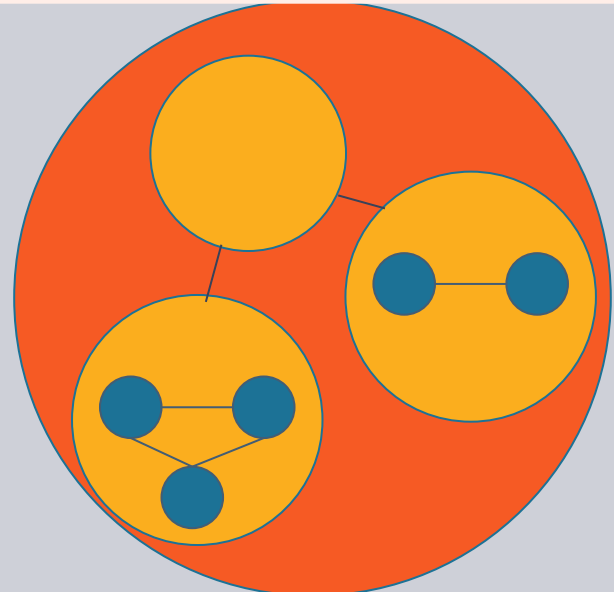
Sistema

Partes do sistema

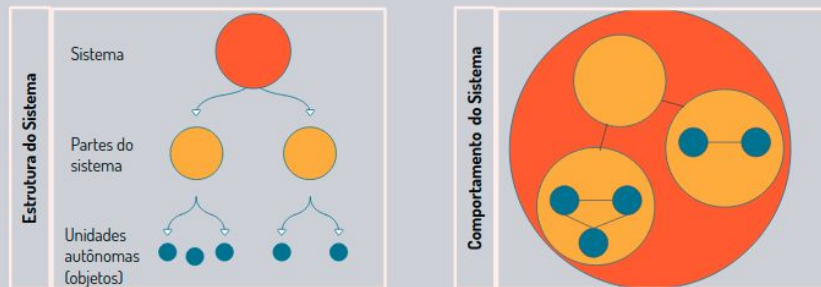
Unidades autônomas (objetos)



Comportamento do Sistema



Paradigma de OO



- **Qualquer sistema pode ser construído a partir de objetos** que **interagem** entre si
- Um objeto pode ser qualquer coisa que seja importante para o sistema
 - Exemplo: célula - sistema do corpo humano; aluno - sistema educativo; roda - carro
- Objetos realizam tarefas **interagindo** com outros objetos
- Cada objeto **pertence** a uma determinada **classe**
- Uma **classe agrupa informação e comportamento de objetos similares**
- Classes são organizadas em **hierarquias**

Paradigma de Desenvolvimento de Software OO

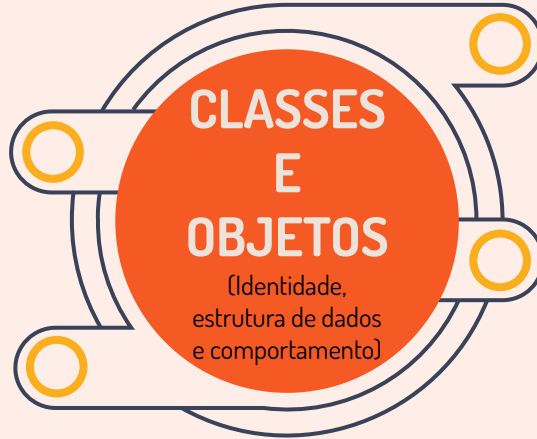
PRINCÍPIOS

Abstração

Módulos, componentes,
classes.

Encapsulamento

Ocultamento de
informações e operações,
Facilidade de mudança



Herança

Reuso de estrutura de dados e
de comportamento . Coesão.

Polimorfismo

Flexibilidade na definição
de métodos. Facilidade de
evolução (incremento)

MODELAGEM – Diagramas UML (Unified Modeling Language)

PROCESSO DE DESENVOLVIMENTO DE SOFTWARE – Etapas de Análise, Design e Implementação

ABSTRAÇÃO

Princípio do Paradigma de
Desenvolvimento de Software
Orientado a Objetos





O que podemos
interpretar dessa obra de
arte?



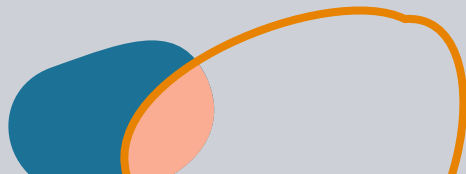
A Caipirinha
De Tarsila do Amaral, 1923
Capivari, SP

O auto retrato **abstrato** faz
referência a origem da artista no
interior de SP



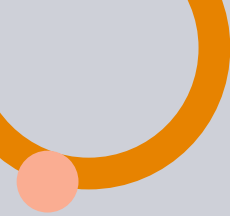
Abstração

Seleção dos aspectos ou características que são relevantes para representar um elemento segundo o interesse do artista (engenheiro de software)

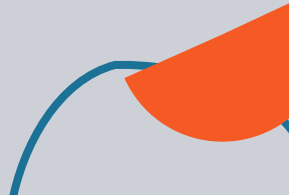


Atividade: Definir a Hierarquia de Abstrações para Representar um Endereço Físico





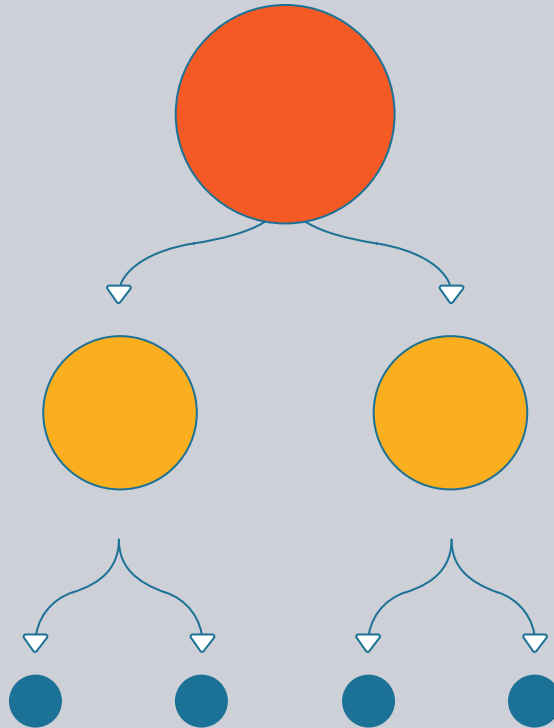
Definir a Hierarquia de Abstrações para Representar um Endereço Físico

- Galaxia
 - Sistema solar
 - Planeta terra
 - Continente
 - Hemisfério
 - Pais
 - Região
 - Estado
 - Cidade
 - Bairro
 - CEP - Rua
 - Número - Coordenada (lat, long)
 - Complemento
- 

Abstrações no Desenvolvimento de Software

Mais abstrato
Menos concreto

Mais concreto
Menos abstrato



Abstrações no Desenvolvimento de Software

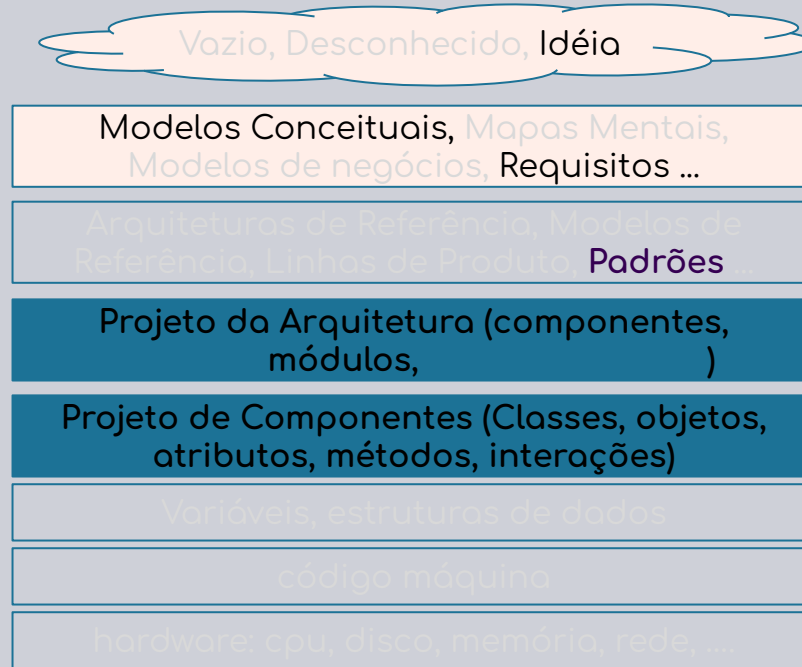


Abstrações no Desenvolvimento de Software

Nosso foco em AP00

+ Abstrato
- Concreto

- Abstrato
+ Concreto



Frameworks de Desenvolvimento, Tecnologias, Bibliotecas, Linguagens de Programação ...

Abstração para Projetar Software

Permite que o projetista (você) se **concentre nos aspectos essenciais de um sistema**, ignorando os detalhes.

Focar na identidade das partes do sistema (módulos, componentes, classes, objetos) ***o que elas fazem***, antes de decidir como implementá-las em código fonte.

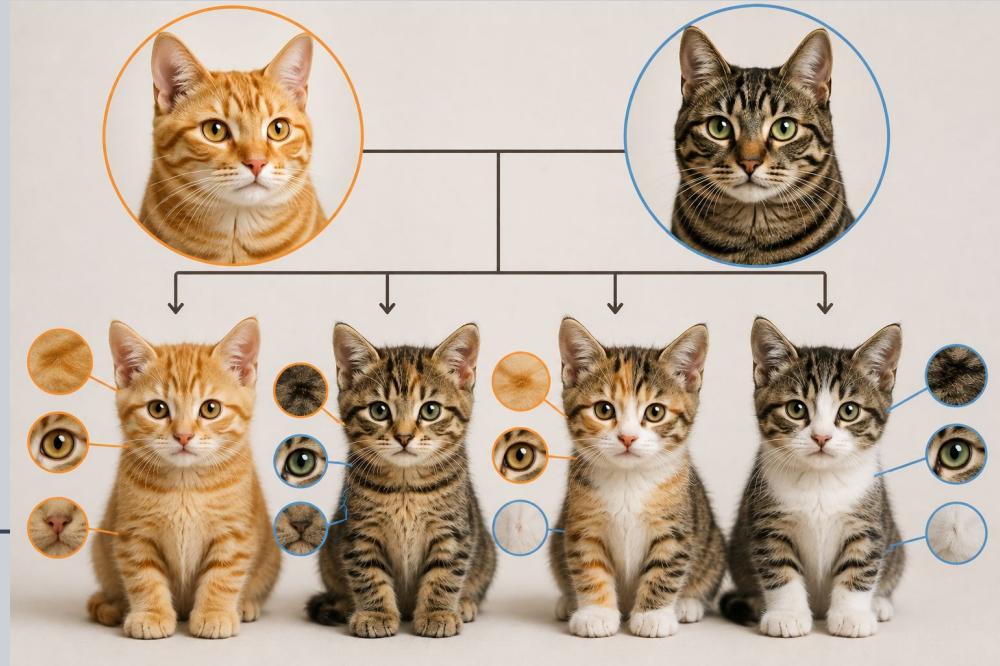
“A capacidade de abstrair aspectos essenciais provavelmente é a habilidade mais importante exigida para desenvolvimento de software orientado a objetos.”

— Blaha and Rumbaugh, 2007



HERANÇA

Princípio do Paradigma de
Desenvolvimento de Software
Orientado a Objetos

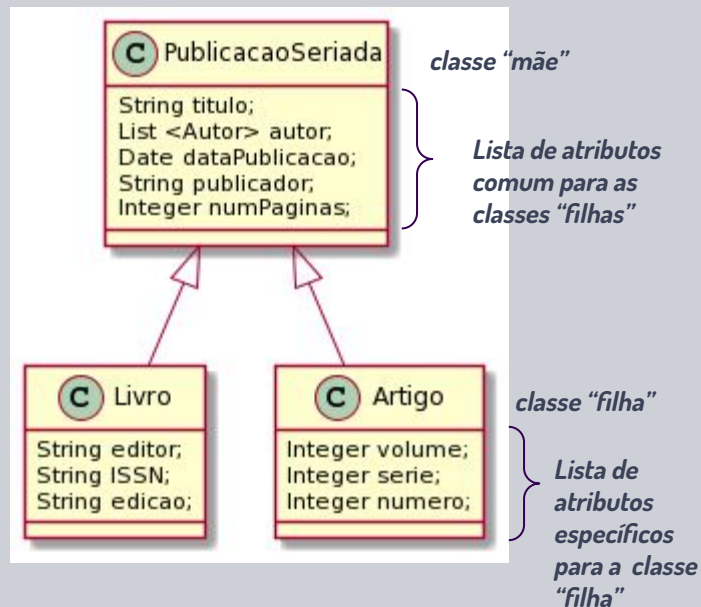


HERANÇA

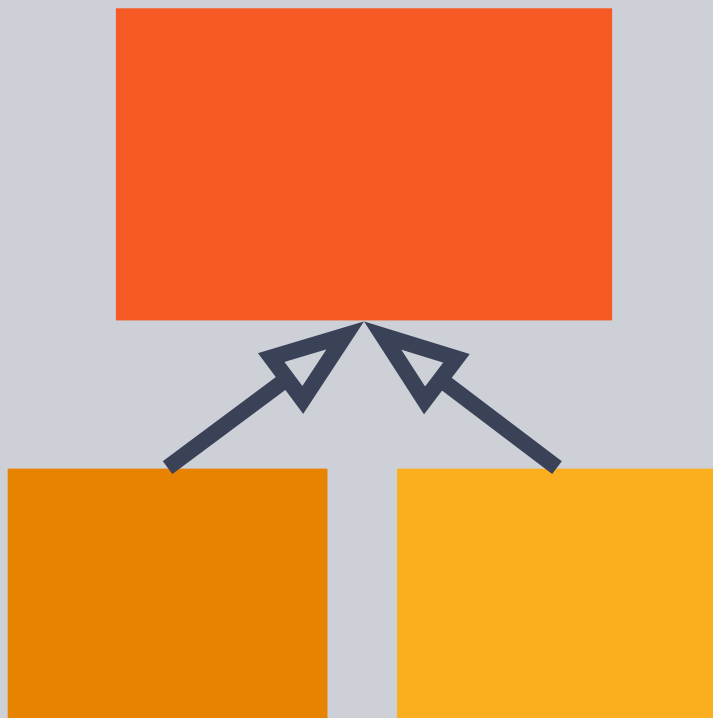
- As classes de elementos derivadas (ou classes filhas) **herdam** atributos e métodos da superclasse (ou classe mãe)
- Superclasses são generalizações ou abstrações
- As classes derivadas são mais específicas ou concretas
- Símbolo de herança: →
- Leitura: **é um**, **herda de**

Livro **é uma** publicacaoSeriada
Artigo **é uma** publicacaoSeriada

classe “filha”



Atividade: Proponha Outro Exemplo de Herança



Exemplos de Herança de Classes

Pessoa

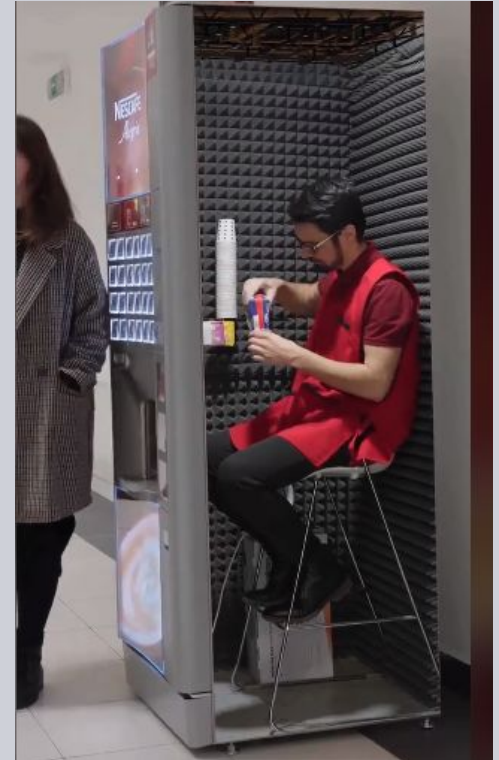
- Adulto
- Menor
- Professor
- Estudante

Veículo

- Carro
- Caminhão
- Moto
- Bike

ENCAPSULAMENTO

Também conhecido como
OCULTAMENTO de informações.



ENCAPSULAMENTO

Permite manter os valores dos atributos dos objetos consistentes.

Mudanças das partes encapsuladas não afetam os objetos que as usam.

As informações de um objeto podem ser somente acessadas através dos métodos definidos na classe

```
public class Livro {  
  
    private final String nome;  
    private final double preco;  
  
    public Livro(String nome, double preco) {  
        this.nome = nome;  
        this.preco = preco;  
    }  
  
    //métodos  
}
```

Exemplo de Encapsulamento

```
public class ContaBancaria {  
    private String nomeTitular;  
    private double saldo;  
  
    public ContaBancaria(String nomeTitular, double saldoInicial) {  
        this.nomeTitular = nomeTitular;  
        this.saldo = saldoInicial;  
    }  
  
    public String getNomeTitular() {  
        return nomeTitular;  
    }  
}
```

```
public void setNomeTitular(String nomeTitular) {  
    this.nomeTitular = nomeTitular;  
}
```

```
public double getSaldo() {  
    return saldo;  
}  
  
public void depositar(double valor) {  
    if (valor > 0) {  
        this.saldo += valor;  
    }  
}  
  
public void sacar(double valor) {  
    if (valor > 0 && valor <= this.saldo) {  
        this.saldo -= valor;  
    }  
}
```



POLIMORFISMO

Princípio do Paradigma de
Desenvolvimento de Software
Orientado a Objetos

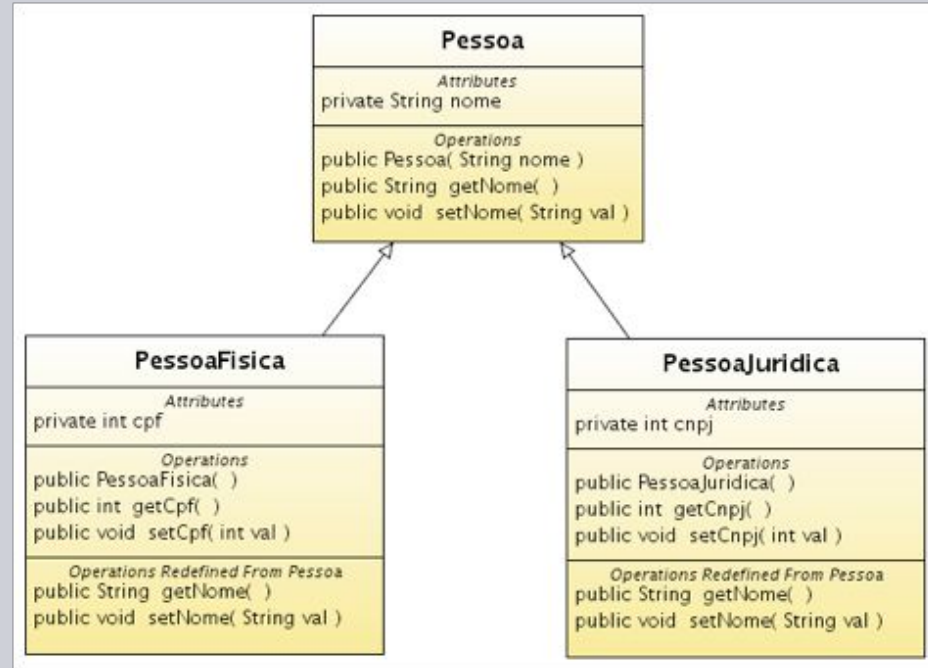


POLIMORFISMO

Capacidade de uma mesma operação se comportar de maneira diferente em classes distintas

Redeclaração de métodos

O sistema verifica se a classe da instância em questão contém o método declarado nela própria ou se o herda de uma superclasse





Exemplo Polimorfismo

```
public class Animal {  
    public void fazerSom() {  
        System.out.println("O animal faz um som.");    }    }  

```

```
public class Cachorro extends Animal {  
    @Override  
    public void fazerSom() {  
        System.out.println("O cachorro faz au au.");    }    }  

```

```
public class Gato extends Animal {  
    @Override  
    public void fazerSom() {  
        System.out.println("O gato faz miau.");    }    }  

```



```
public class Calculadora {  
    public int somar(int a, int b) {  
        return a + b;  
    }  
  
    public double somar(double a, double b) {  
        return a + b;  
    }  
  
    public int somar(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

Outro Exemplo

Paradigma de Desenvolvimento de Software OO

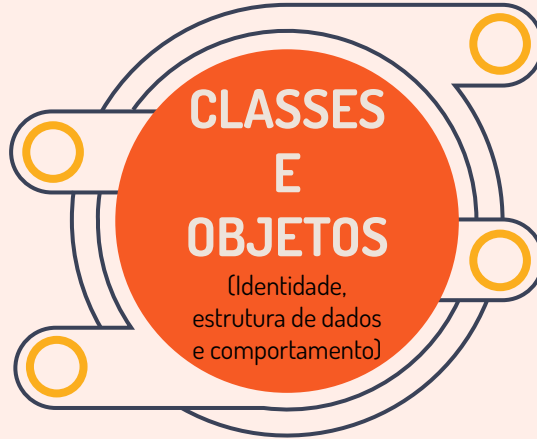
PRINCÍPIOS

Abstração

Módulos, componentes,
classes.

Encapsulamento

Ocultamento de
informações e operações,
Facilidade de mudança



Herança

Reuso de estrutura de dados e
de comportamento . Coesão.

Polimorfismo

Flexibilidade na definição
de métodos. Facilidade de
evolução (incremento)

MODELAGEM – Diagramas UML (Unified Modeling Language)

PROCESSO DE DESENVOLVIMENTO DE SOFTWARE – Etapas de Análise, Design e Implementação



Referências

Michael Blaha and James Rumbaugh. **Modelagem e Projetos baseados em Objetos com UML 2**. 2a Ed. Elsevier. 2006

Gilleanes T. A. Guedes. **UML 2: uma abordagem prática**. 3a Edição. Novatec (Ed) 2018

Hélio Engholm Jr. **Análise e Design Orientado a Objetos**. 1a Ed. Novatec. 2013.

